

CO4- AT2 - Lab Practice

R kalyan kumar

192472274

CSA0905-Java Programming

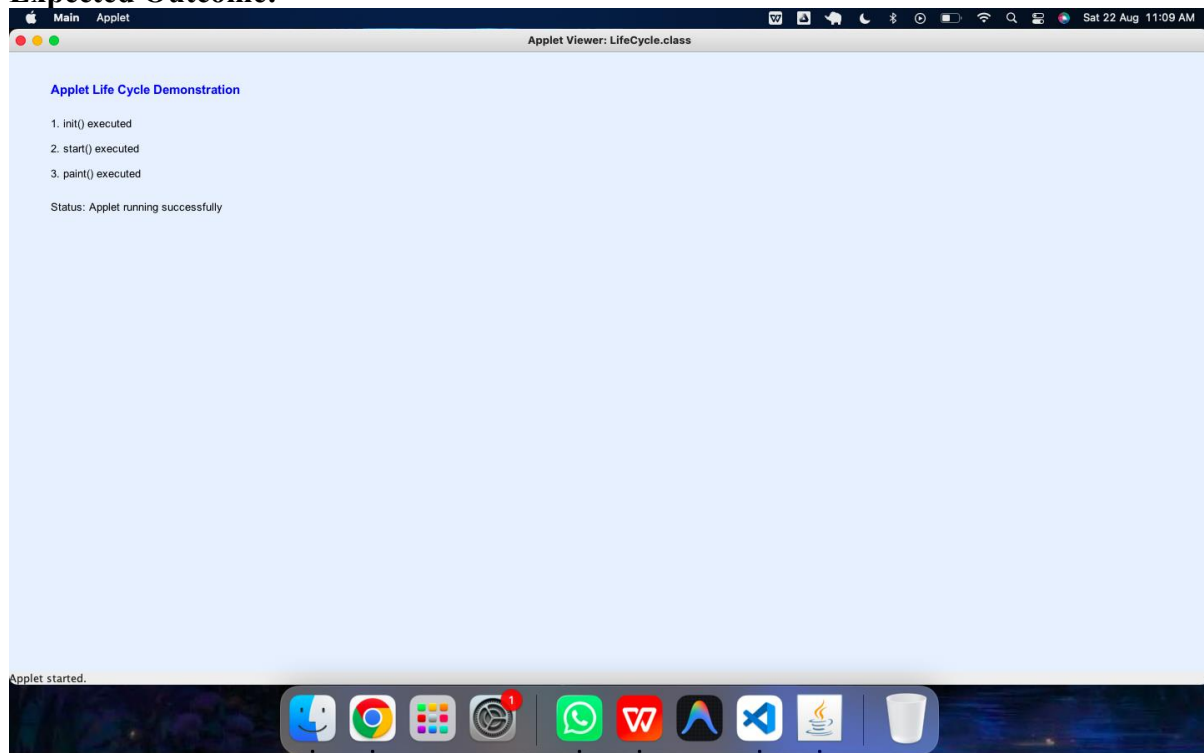
Java Lab Practice – Applets, AWT Components and Layout Managers

Question 1 – Applet Life Cycle

Develop a Java Applet to demonstrate the **Applet Life Cycle** methods: `init()`, `start()`, `paint()`, `stop()`, and `destroy()`.

Display a suitable message in the applet window whenever each life-cycle method is invoked. Observe the sequence of execution by running the applet and interacting with it.

Expected Outcome:



Question 2 – Generating Graphic Images Using Applet

Develop a Java Applet that uses the **Graphics class** to generate a simple graphical scene.

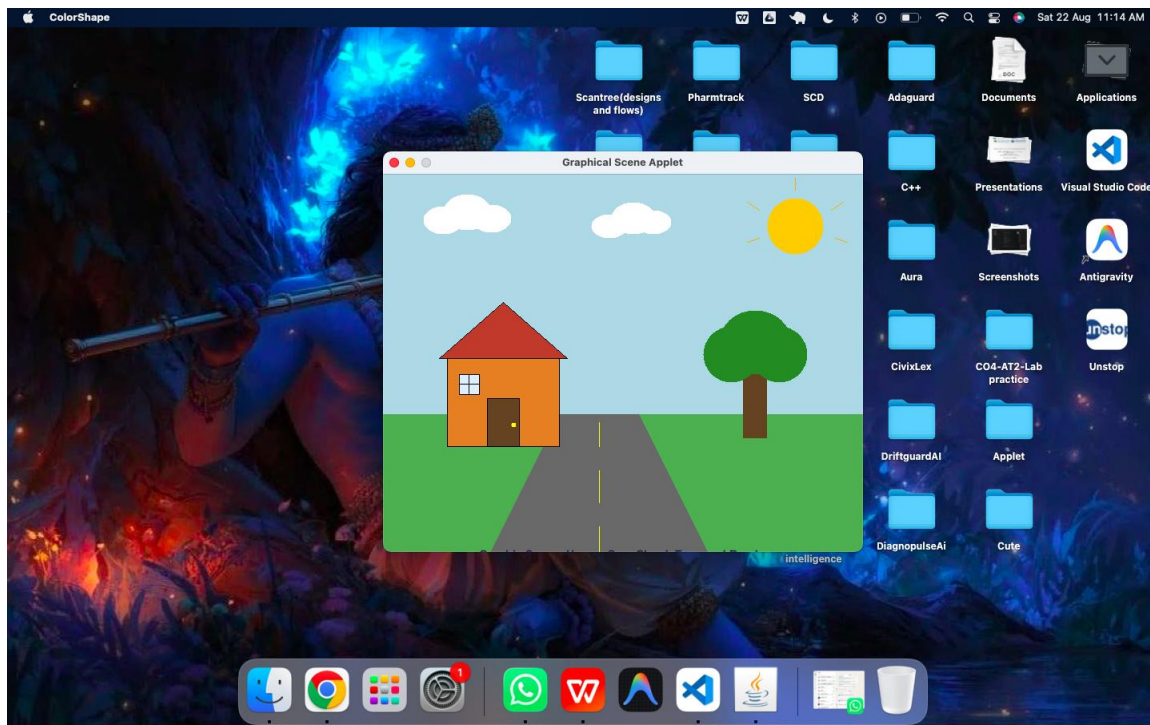
The applet should draw:

- A house
- Sun
- Trees
- Road
- Clouds

Use methods such as `drawLine()`, `drawRect()`, `drawOval()`, `fillRect()`, `fillOval()`, and appropriate colors.

Expected Outcome:

Students should demonstrate the use of the Graphics class for drawing shapes and images.



Question 3 – Calculator Application Using Applet Parameters

Develop a **Calculator Applet** that accepts two numbers and an arithmetic operator through **HTML/Applet parameters**.

The applet should perform:

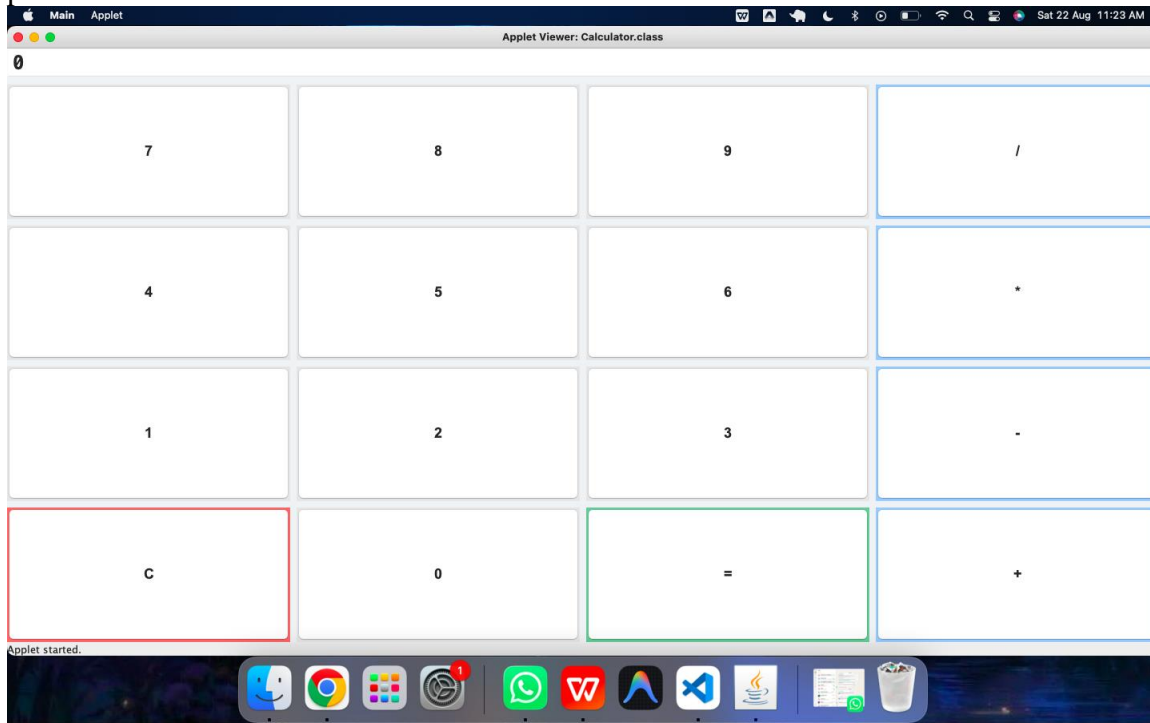
- Addition

- Subtraction
- Multiplication
- Division

Use the `getParameter()` method to retrieve the values passed to the applet.

Expected Outcome:

Students should understand how parameters are passed from an HTML page to an Applet and processed in Java.



Question 4 – Student Profile Using AWT Components

Develop a **Student Profile Application** using Java AWT components such as:

- Label
- TextField
- Checkbox
- CheckboxGroup
- Choice
- TextArea
- Button
- List

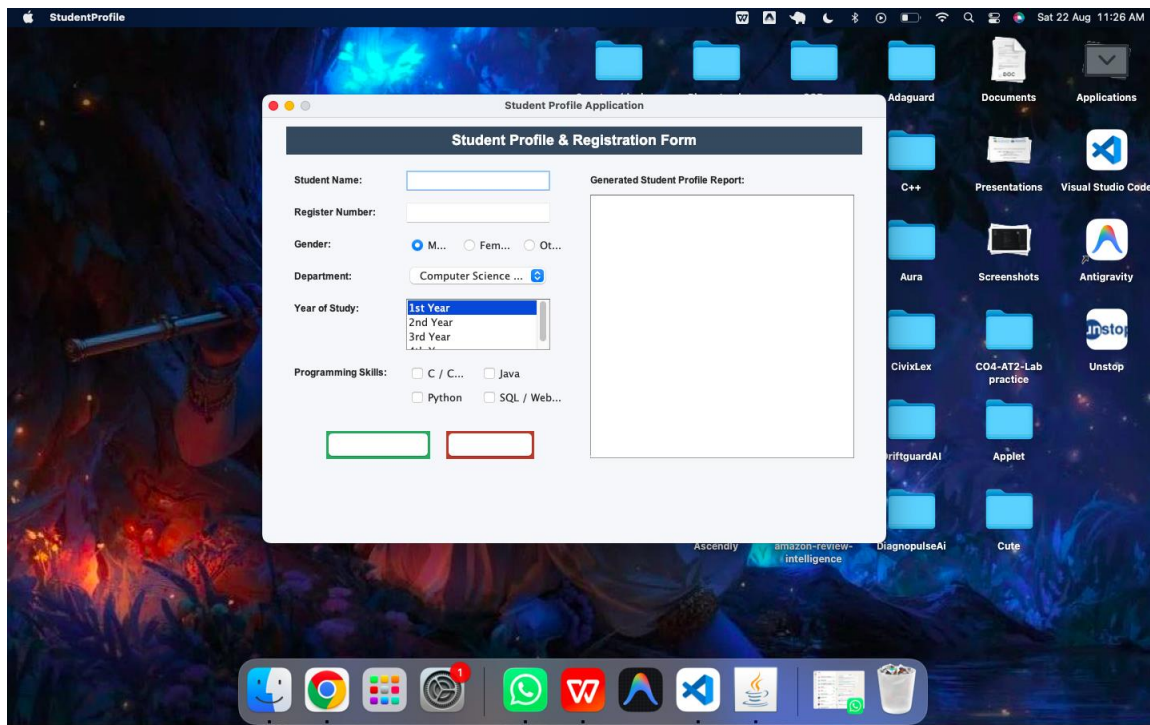
The application should accept student details such as:

- Name
- Register Number
- Gender
- Department
- Year
- Skills/Programming Languages

When the **Generate Report** button is clicked, display the entered student information in a List or TextArea.

Expected Outcome:

Students should demonstrate the use of multiple AWT components and event handling.



Question 5 – GridLayout with Event Handling

Develop a Java AWT application using **GridLayout** to create a simple form or calculator interface.

Arrange the components in a grid and implement appropriate **event handling** using ActionListener.

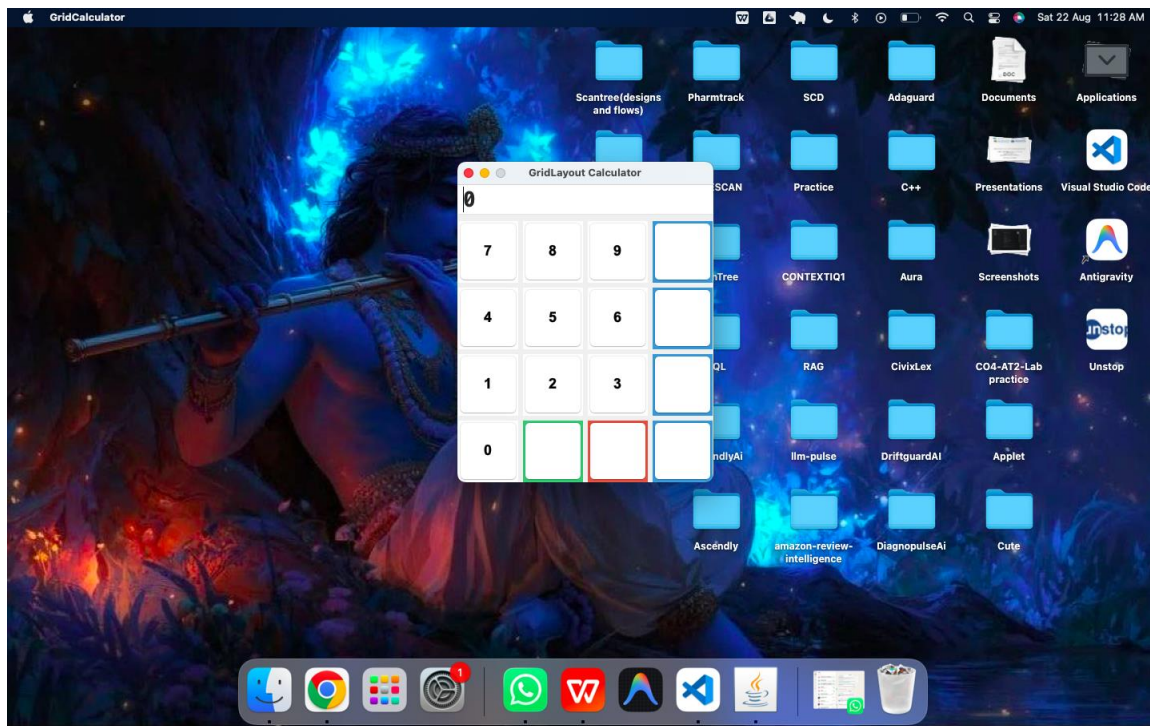
For example, create a calculator with buttons:

7 8 9 +
4 5 6 -
1 2 3 *
0 = C /

Perform the corresponding operation when a button is clicked.

Expected Outcome:

Students should understand GridLayout and button-based event handling.



Question 6 – GridBagLayout with Event Handling

Develop a Java AWT **Student Registration Form** using GridBagLayout.

The form should contain:

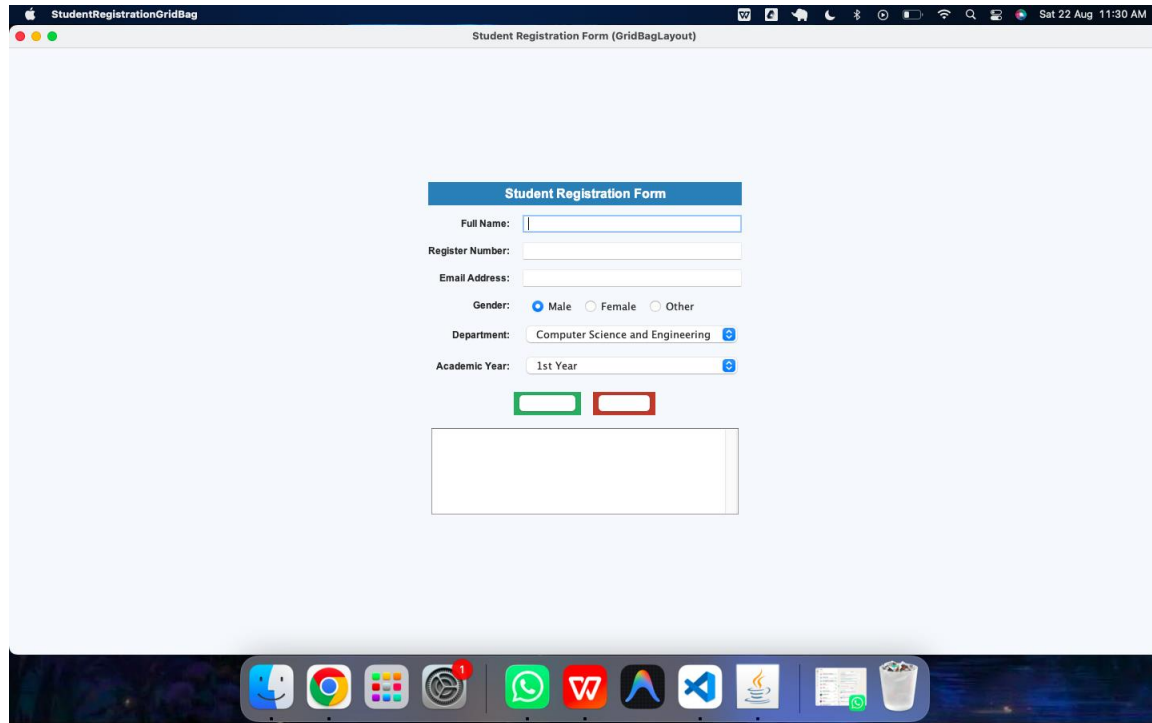
- Name
- Register Number
- Department
- Email
- Gender
- Year
- Submit Button
- Reset Button

Use GridBagConstraints to properly position and align the components.

Implement event handling for the **Submit** and **Reset** buttons.

Expected Outcome:

Students should understand flexible component positioning using GridBagLayout and event handling.



Question 7 – BorderLayout with Event Handling

Develop a Java AWT application using **BorderLayout**.

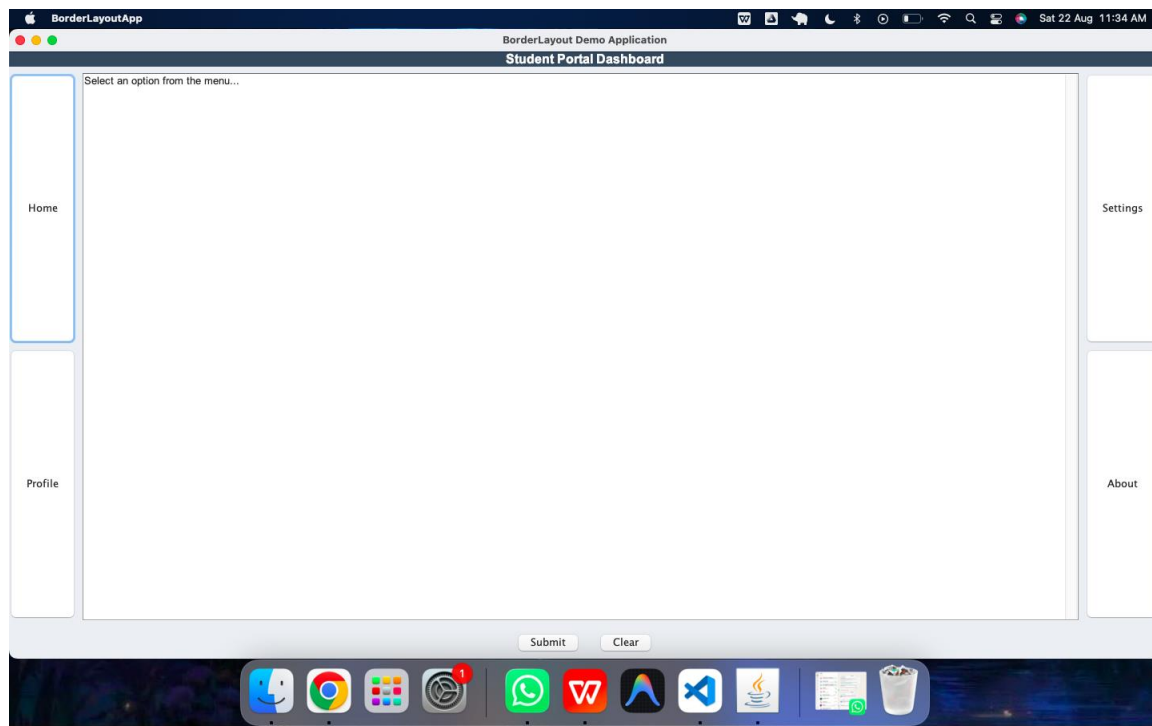
Divide the window into:

- NORTH – Header
- SOUTH – Buttons
- EAST – Options
- WEST – Menu
- CENTER – Display area

Add buttons in the interface and implement event handling so that clicking different buttons changes the content displayed in the center area.

Expected Outcome:

Students should understand the five regions of BorderLayout and event-driven programming.



Question 8 – CardLayout with Event Handling

Develop a Java AWT application using **CardLayout** to create a multi-page interface.

Create at least three cards:

- **Card 1:** Student Information
- **Card 2:** Academic Information
- **Card 3:** Confirmation/Report

Provide **Next**, **Previous**, and **Submit** buttons to navigate between the cards.

Implement event handling to switch between cards using CardLayout.

Expected Outcome:

Students should understand how CardLayout can be used to create multiple screens within a single window.

